

Wand AI - Engineering Challenges

Overview

We're an AI company building systems that let enterprises use AI as part of their workforce in the hybrid workforce model.

You will complete **one** of the following challenges **solo** in 24 hours, focusing on your specialty

You should treat this like a real-world scenario: requirements are high-level, trade-offs must be made, and you are responsible for **design, implementation, and documentation**.

Challenge 1:

Multi-Agent Task Solver

Core Concept

Build a system that accepts a high-level business request in plain language (e.g., "*Summarize the last 3 quarters' financial trends and create a chart*"), then uses **multiple specialized AI agents** to break down the task, execute subtasks, and return a final structured result.

Core Requirements (Common for All Roles)

1. **Input:** User enters a business request in text.
2. **Planning:** The system decides which agents are needed and what each should do.
3. **Execution:** Agents complete their tasks and share results.
4. **Aggregation:** The system combines results into a final answer.
5. **Visibility:** User sees progress/status updates for each agent.

Expectations

- Design agents with:
 - Clear role definitions (specialized prompts).
 - Context sharing between agents.
 - Tool usage (e.g., call a Python code executor, search API).
- Demonstrate orchestration logic that avoids common LLM pitfalls (hallucination, repetition).
- **High Marks:** Agents can handle ambiguous or incomplete tasks by asking clarifying questions.

Stretch Goals

- Add a “live conversation” mode where the user can chat with the orchestrator mid-execution.
- Support multi-turn refinement (user modifies request after first output).

Challenge 2:

AI-Powered Knowledge Base Search & Enrichment

Core Concept

Build a system where a user can upload multiple documents, search them in natural language, get AI-generated answers, and **flag when the answer is incomplete** — then suggest ways to enrich the knowledge base.

Core Requirements (Common for All Roles)

1. **Document Upload & Storage.**
2. **Search:** Users can ask a question in plain language.
3. **Answer:** AI generates an answer using the documents.
4. **Completeness Check:** AI detects when information is missing or uncertain.
5. **Enrichment Suggestion:** Suggest additional documents, data, or actions to fill the gap.

Expectations

- Build the RAG (Retrieval-Augmented Generation) pipeline.
- Prompt LLM to:
 - Use retrieved documents for answers.
 - Detect when it lacks enough info.
 - Suggest enrichment strategies.
- **High Marks:** Uses structured output (JSON with **answer**, **confidence**, **missing_info**) and gracefully handles irrelevant documents.

Stretch Goals

- Add “auto-enrichment” where the system fetches missing data from a trusted external source. Allow users to rate answer quality to improve the pipeline.

General Deliverables

- **Working prototype** (local or hosted).
- **README** describing:
 - Design decisions.
 - Trade-offs made due to the 24h constraint.
 - How to run/test the system.
- **Short Loom or screen recording** demo (max 5 min).
- Code in a public or private GitHub repo.